

Unit 2: Graph Algorithms

Comprehensive Study Notes

Algorithms & Data Structures

Exam-Level Reference Material

Complete with Proofs, Visual Walkthroughs & Complexity Analysis

Table of Contents

Chapter 0: Introduction to Graphs

- 0.1 Fundamental Definitions
- 0.2 Examples and Applications
- 0.3 Paths, Connectivity, and Trees
- 0.4 Key Theorems on Trees

Chapter 1: Graph Representations

- 1.1 Adjacency Matrix
- 1.2 Adjacency List
- 1.3 Degree and the Handshaking Lemma

Chapter 2: Breadth-First Search (BFS)

- 2.1 Intuition and Layers
- 2.2 Algorithm and Working
- 2.3 Visual Walkthrough
- 2.4 Full Complexity Derivation
- 2.5 Proof of Correctness
- 2.6 The BFS Tree and Nontree Edges

Chapter 3: Depth-First Search (DFS)

- 3.1 Intuition and the Maze Analogy
- 3.2 Algorithm and Working
- 3.3 Visual Walkthrough
- 3.4 Full Complexity Derivation
- 3.5 Structural Theorems on DFS Trees

Chapter 4: Connected Components

- 4.1 The Generic Component-Growing Algorithm

4.2 Finding All Connected Components

4.3 The Disjointness Property

Chapter 5: Testing Bipartiteness

5.1 Definition and 2-Coloring

5.2 BFS-Based Algorithm

5.3 Visual Walkthrough

5.3 Theorems on Bipartiteness

Chapter 6: Directed Graphs

6.1 Representing Directed Graphs

6.2 BFS and DFS in Directed Graphs

6.3 Strong Connectivity

6.4 Strong Components

Chapter 7: DAGs and Topological Ordering

7.1 Directed Acyclic Graphs

7.2 Topological Ordering

7.3 Algorithm and Visual Walkthrough

Chapter 8: Dijkstra's Algorithm

8.1 Overview and Invariant

8.2 Algorithm and Working

8.3 Visual Walkthrough

8.4 Full Complexity Derivation

8.5 Proof of Correctness

Chapter 9: Minimum Spanning Trees

9.1 The MST Problem

9.2 The Cut Property

9.3 The Cycle Property

9.4 Prim's Algorithm

9.5 Reverse-Delete Algorithm

Summary: Key Facts and Complexity Table

Chapter 0: Introduction to Graphs

0.1 Fundamental Definitions

Definition: Graph (Undirected)

A **graph** G is a mathematical structure encoding **pairwise relationships** among a set of objects. Formally, $G = (V, E)$ where:

- V is a set of **nodes** (also called **vertices** — the terms are synonymous).
- E is a set of **edges**, where each edge $e \in E$ is a two-element subset $\{u, v\}$ for some $u, v \in V$.

The nodes u and v are called the **ends** of edge e . Edges encode a **symmetric relationship**: if $\{u, v\} \in E$, the relationship is mutual.

Notation warning: Although an undirected edge is technically a *set* $\{u, v\}$, it is commonly written as an ordered pair (u, v) . For undirected graphs, (u, v) and (v, u) refer to the exact same edge.

Definition: Directed Graph (Digraph)

A **directed graph** $G' = (V, E')$ models **asymmetric** relationships. Each directed edge $e' \in E'$ is an **ordered pair** (u, v) where:

- u is the **tail** of the edge (the edge **leaves** u).
- v is the **head** of the edge (the edge **enters** v).

Because the pair is ordered, (u, v) and (v, u) are **distinct** edges.

Default convention: When the term "graph" is used without qualification, it refers to an **undirected graph**.

Paths and Connectivity

Definition: Path

A **path** in an undirected graph is a sequence of nodes $P = v_1, v_2, \dots, v_k$ where each consecutive pair (v_i, v_{i+1}) is joined by an edge in E .

Definition: Simple Path

A path is **simple** if all its vertices are distinct (no node appears more than once).

Definition: Cycle

A **cycle** is a path $v_1, v_2, \dots, v_{k-1}, v_k$ satisfying:

1. $k > 2$ (at least three edges, preventing degenerate back-and-forth).
2. The first $k - 1$ nodes are all distinct.
3. $v_1 = v_k$ (the sequence cycles back to its start).

Definition: Connected Graph

An undirected graph is **connected** if for **every pair of nodes** u and v , there exists a path from u to v .

Definition: Strongly Connected (Directed)

A directed graph is **strongly connected** if for **every two nodes** u and v , there is a directed path from u to v **and** from v to u .

Definition: Distance

The **distance** between two nodes u and v is the **minimum number of edges** in a u - v path. If $u = v$, the distance is 0; if no path exists, the distance is ∞ .

0.2 Examples and Applications

Graphs model an enormous variety of real-world systems. For each application, understand **what nodes represent**, **what edges represent**, and **whether the graph is directed or undirected**.

Table 1 Major Categories of Graph Applications

Category	Nodes	Edges	Directed?
Transportation (Airlines)	Airports	Nonstop flights	Often undirected (bidirectional)
Communication (Internet)	Computers / ISPs	Physical links / Peering	Both
Information (WWW)	Web pages	Hyperlinks	Directed (crucial)
Social Networks	People	Friendship / Advice- seeking	Both
Dependency (Courses)	Courses	Prerequisites	Directed
Ecological Food Webs	Species	Consumption	Directed

0.3 Trees

Definition: Tree

An undirected graph is a **tree** if it satisfies two conditions:

1. It is **connected**.
2. It does **not contain a cycle**.

Key property: Deleting **any** edge from a tree disconnects it. Trees have no "redundant" edges — there is exactly one simple path between any two nodes.

Definition: Rooted Tree

A **rooted tree** is obtained by selecting one node r as the **root** and orienting each edge away from r . For every other node $v \neq r$:

- The **parent** of v is the unique node u that directly precedes v on the path from r to v .
- w is a **child** of v if v is the parent of w .
- w is a **descendant** of v (and v is an **ancestor** of w) if v lies on the unique path from r to w .
- A **leaf** is a node with no children.

0.4 Key Theorems on Trees

Theorem 3.1: Tree Edge Count

Every n -node tree has exactly $n - 1$ edges.

Proof (by Induction on n)

Base Case ($n = 1$): A tree with one node has 0 edges. Since $1 - 1 = 0$, the theorem holds.

Inductive Step: Assume the theorem holds for all trees with k nodes where $k < n$. Consider a tree T with n nodes.

Since T is a tree, it is connected and acyclic. Removing any edge from T disconnects it into exactly **two components** (there are no cycles, so every edge is a bridge). Let these components be T_1 and T_2 with n_1 and n_2 nodes respectively, where $n_1 + n_2 = n$ and $n_1, n_2 \geq 1$.

Each component is itself a tree (connected and acyclic). By the induction hypothesis:

$$|E(T_1)| = n_1 - 1 \quad \text{and} \quad |E(T_2)| = n_2 - 1$$

The total number of edges in T is:

$$|E(T)| = (n_1 - 1) + (n_2 - 1) + 1 = n_1 + n_2 - 1 = n - 1$$

The "+1" accounts for the edge we removed. Thus T has exactly $n - 1$ edges. ■

Theorem 3.2: Equivalence of Tree Properties

Let G be an undirected graph on n nodes. Any two of the following statements implies the third:

- (i) G is connected.
- (ii) G does not contain a cycle.
- (iii) G has $n - 1$ edges.

Proof**Case 1: (i) $\wedge$ (ii) $\Rightarrow$ (iii)**

If G is connected and acyclic, then by definition G is a tree. By Theorem 3.1, $|E(G)| = n - 1$. ■

Case 2: (i) $\wedge$ (iii) $\Rightarrow$ (ii)

Assume G is connected and has $n - 1$ edges. Suppose for contradiction that G contains a cycle. Remove one edge from this cycle. The graph remains connected (removing an edge from a cycle does not disconnect). Continue removing edges from cycles until no cycles remain. We obtain a connected, acyclic graph T' with n nodes. By Theorem 3.1, T' has $n - 1$ edges. But we removed at least one edge, so T' has at most $n - 2$ edges. **Contradiction.** Therefore G contains no cycle. ■

Case 3: (ii) $\wedge$ (iii) $\Rightarrow$ (i)

Assume G is acyclic and has $n - 1$ edges. Suppose for contradiction that G is disconnected. Then G has $k \geq 2$ connected components $C_1, C_2, \dots, C_k$ with $n_1, n_2, \dots, n_k$ nodes respectively, where $\sum n_i = n$.

Each component C_i is connected and acyclic (since G is acyclic), so each C_i is a tree. By Theorem 3.1, each C_i has $n_i - 1$ edges. The total number of edges is:

$$|E(G)| = \sum_{i=1}^k (n_i - 1) = \sum_{i=1}^k n_i - k = n - k$$

Since $k \geq 2$, we have $|E(G)| \leq n - 2$, contradicting the assumption that G has $n - 1$ edges. **Contradiction.** Therefore G must be connected. ■

Chapter 1: Graph Representations

Before implementing graph algorithms, we must decide how to represent a graph in memory. We use two fundamental parameters:

- $n = |V|$: number of nodes.
- $m = |E|$: number of edges.

For a simple undirected graph, the maximum number of edges is $\binom{n}{2} = \frac{n(n-1)}{2} = O(n^2)$. If the graph is connected, $m \geq n - 1$.

1.1 Adjacency Matrix Representation

Definition: Adjacency Matrix

Assume nodes are numbered $V = \{1, 2, \dots, n\}$. The **adjacency matrix** is an $n \times n$ matrix A where:

$$A[u, v] = \begin{cases} 1 & \text{if edge } (u, v) \in E \\ 0 & \text{otherwise} \end{cases}$$

For undirected graphs, A is **symmetric**: $A[u, v] = A[v, u]$.

Table 2 Adjacency Matrix: Trade-offs

Aspect	Performance	Analysis
Edge lookup	$O(1)$	Direct array access $A[u, v]$
Space	$\Theta(n^2)$	One entry per ordered pair of nodes
Neighbor enumeration	$\Theta(n)$	Must scan all n possible neighbors of u

Disadvantage: For sparse graphs where $m \ll n^2$, the adjacency matrix wastes space and time scanning mostly-zero rows.

1.2 Adjacency List Representation

Definition: Adjacency List

In the **adjacency list** representation, we maintain an array Adj of length n , where $Adj[v]$ is a list containing all nodes adjacent to v .

Table 3 Adjacency List: Trade-offs

Aspect	Performance	Analysis
Space	$O(m + n)$	n array pointers + $2m$ total list entries (undirected)
Edge lookup	$O(\deg(u))$	Scan $Adj[u]$ for target v
Neighbor enumeration	$O(1)$ per neighbor	Direct list iteration

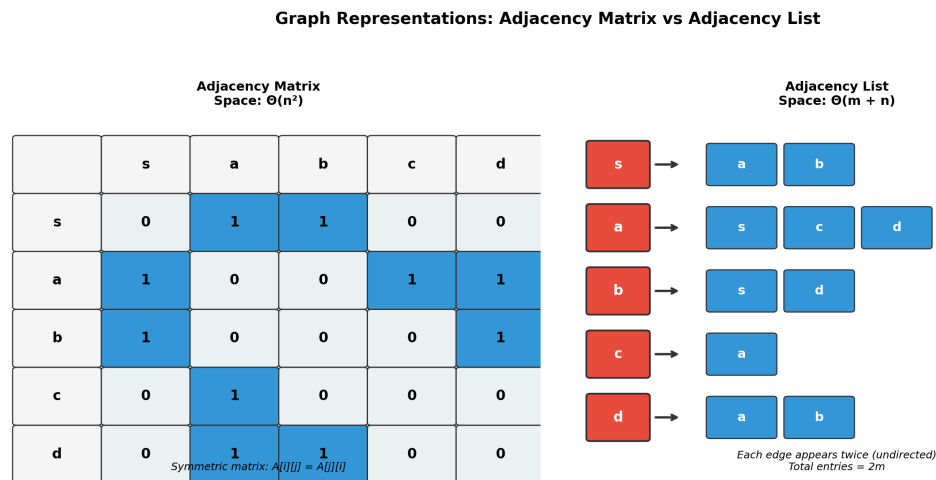


Figure 1: Visual comparison of adjacency matrix and adjacency list representations for the same graph. The matrix uses n^2 entries regardless of edge count, while the list uses only $2m$ entries for undirected graphs.

Theorem 3.10: Space Complexity of Representations

The adjacency matrix requires $O(n^2)$ space; the adjacency list requires $O(m + n)$ space.

Proof

Adjacency Matrix: The matrix has n rows and n columns, giving n^2 entries. Each entry is $O(1)$ space. Total: $O(n^2)$.

Adjacency List: We need an array of n pointers: $O(n)$. Each undirected edge $\{u, v\}$ appears in exactly two lists: $v \in Adj[u]$ and $u \in Adj[v]$. Thus total list entries = $2m = O(m)$. Total: $O(m + n)$. ■

1.3 Degree and the Handshaking Lemma

Definition: Degree

The **degree** of a node v , denoted n_v or $\deg(v)$, is the number of incident edges it has. In the adjacency list representation, $\deg(v) = |Adj[v]|$.

Theorem 3.9: The Handshaking Lemma

$$\sum_{v \in V} \deg(v) = 2m$$

Proof

Consider the sum of degrees over all nodes. Each edge $e = \{v, w\}$ contributes exactly 1 to $\deg(v)$ and exactly 1 to $\deg(w)$. Therefore, each edge contributes exactly 2 to the total sum of degrees.

Summing over all m edges:

$$\sum_{v \in V} \deg(v) = 2m$$

This identity is fundamental for proving linear-time bounds on graph algorithms. ■

Chapter 2: Breadth-First Search (BFS)

2.1 Intuition and Layers

Breadth-First Search (BFS) is the simplest algorithm for determining **s-t connectivity**: given a graph G and two nodes s (source) and t (target), is there a path from s to t ?

Physical analogy: Imagine the graph as a **maze** where nodes are rooms and edges are hallways. BFS corresponds to "flooding" the maze with an expanding wave starting at s . The wave grows outward uniformly, visiting all reachable nodes layer by layer.

Definition: BFS Layers

BFS constructs successive layers $L_0, L_1, L_2, \dots$:

- **Layer L_0 :** $\{s\}$ (just the starting node).
- **Layer L_1 :** All neighbors of s .
- **Layer L_{j+1} :** All nodes that (a) do not belong to any earlier layer, and (b) have an edge to at least one node in L_j .

Crucial connection to distance: Layer L_j is precisely the set of all nodes at **distance exactly j** from s . BFS simultaneously computes shortest paths (in terms of edge count) from s to every reachable node.

2.2 Algorithm and Working

Algorithm: BFS(s)

```

BFS(s):
    Set Discovered[s] = true and Discovered[v] = false for all other v
    Initialize L[0] to consist of the single element s
    Set the layer counter i = 0
    Set the current BFS tree T = ∅
    While L[i] is not empty
        Initialize an empty list L[i + 1]
        For each node u ∈ L[i]
            Consider each edge (u, v) incident to u
            If Discovered[v] = false then
                Set Discovered[v] = true
                Add edge (u, v) to the tree T
                Add v to the list L[i + 1]
            Endif
        Endfor
        Increment the layer counter i by one
    Endwhile

```

Initialization: The source node s is marked as discovered. All other nodes are undiscovered. Layer $L[0]$ contains only s . The BFS tree T starts empty.

Main Loop: The algorithm processes layers iteratively:

1. For each node u in the current layer $L[i]$, examine all edges (u, v) incident to u .
2. If v has not been discovered, mark it discovered, add edge (u, v) to the BFS tree T , and place v in the next layer $L[i + 1]$.
3. Once all nodes in $L[i]$ are processed, increment i and continue until a layer is empty.

Key Insight: Nodes are discovered in order of their distance from s . The BFS tree T contains exactly one path from s to each discovered node, and this path is a **shortest path** in the unweighted graph.

2.3 Visual Walkthrough

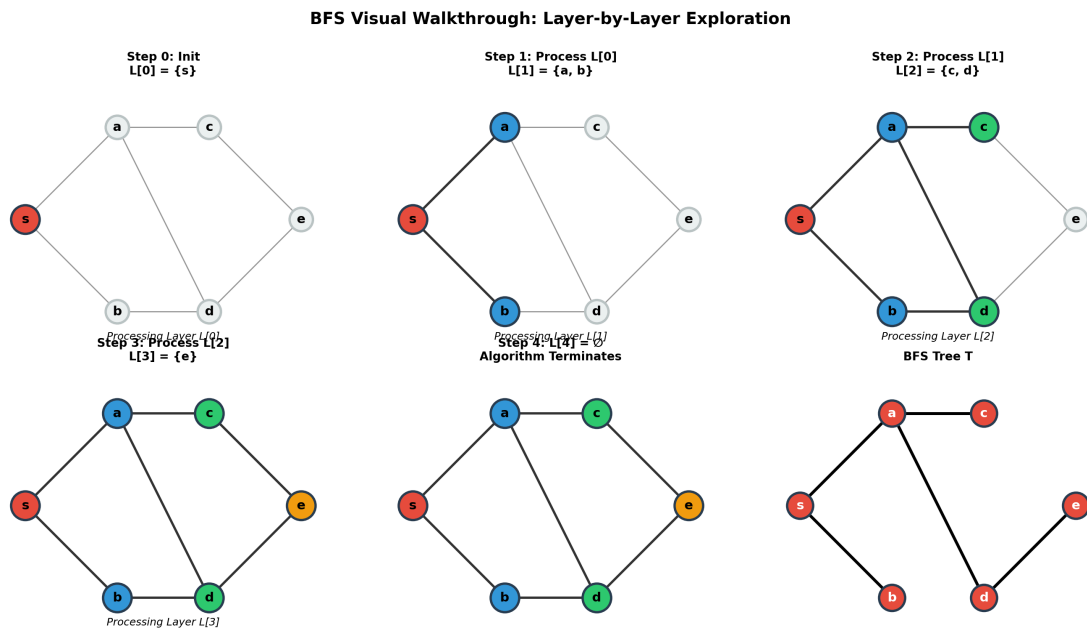


Figure 2: BFS layer-by-layer exploration starting from node s . Each step shows the newly discovered nodes (colored by layer) and the final BFS tree (rightmost panel). Red = Layer 0, Blue = Layer 1, Green = Layer 2, Orange = Layer 3.

Execution trace on the example graph:

Table 4 BFS Execution Trace

Step	Action	$L[0]$	$L[1]$	$L[2]$	$L[3]$	Discovered
Init	Set Discovered[s] = true	{s}	$\emptyset$	$\emptyset$	$\emptyset$	{s}
i=0	Process s: neighbors a, b discovered	{s}	{a, b}	$\emptyset$	$\emptyset$	{s, a, b}
i=1	Process a: discovers c, d. Process b: d already discovered	{s}	{a, b}	{c, d}	$\emptyset$	{s, a, b, c, d}
i=2	Process c: neighbor a already known. Process d: discovers e	{s}	{a, b}	{c, d}	{e}	{s, a, b, c, d, e}
i=3	Process e: neighbors c, d already known. $L[4] = \emptyset$	{s}	{a, b}	{c, d}	{e}	All nodes
i=4	$L[4]$ is empty	Algorithm terminates				

2.4 Full Complexity Derivation

Theorem 3.11: BFS Time Complexity

The above implementation of BFS runs in time $O(m + n)$ if the graph is given by adjacency list representation.

Step-by-Step Derivation

Step 1: Initialization

Setting `Discovered` for all n nodes takes $O(n)$ time. Initializing $L[0]$ and T takes $O(1)$.

Step 2: Main Loop — Node Processing

Each node u is placed in exactly one layer $L[i]$ and is processed exactly once (after being marked `Discovered`, it is never processed again).

Step 3: Edge Examination

When processing node u , the algorithm examines all edges incident to u by iterating through $Adj[u]$. The number of edges incident to u is $\deg(u)$.

The total time spent examining all edges is:

$$T_{\text{edges}} = \sum_{u \in V} \deg(u) = 2m \quad (\text{by Theorem 3.9, the Handshaking Lemma})$$

Step 4: Constant-Time Operations

Each discovery event involves: marking `Discovered[v] = true` ($O(1)$), adding edge to T ($O(1)$), adding v to layer list ($O(1)$). Since each node is discovered at most once, total: $O(n)$.

Step 5: Total Time

$$T_{\text{total}} = T_{\text{init}} + T_{\text{edges}} + T_{\text{ops}} = O(n) + O(2m) + O(n) = O(m + n)$$

For connected graphs, $m \geq n - 1$, so $O(m + n)$ is equivalent to $O(m)$. This is called **linear time** because it is linear in the input size ($\Theta(m + n)$ bits to represent the adjacency list). ■

2.5 Proof of Correctness

Theorem 3.3: BFS Layer Correctness

For each $j \geq 1$, layer L_j produced by BFS consists of all nodes at distance exactly j from s . There is a path from s to t if and only if t appears in some layer.

Proof (by Induction on j)

Part 1: L_j contains exactly nodes at distance j from s .

Base Case ($j = 0$): $L[0] = \{s\}$. The distance from s to itself is 0. No other node is at distance 0. Correct.

Inductive Step: Assume $L[i]$ contains exactly all nodes at distance i from s . We show $L[i + 1]$ contains exactly all nodes at distance $i + 1$ from s .

($\subseteq$) Let $v \in L[i + 1]$. Then v was discovered from some $u \in L[i]$ where $(u, v) \in E$ and v was undiscovered. By induction, $\text{dist}(s, u) = i$. Therefore there exists a path of length i from s to u , and adding edge (u, v) gives a path of length $i + 1$ from s to v . So $\text{dist}(s, v) \leq i + 1$.

Could $\text{dist}(s, v) < i + 1$? If $\text{dist}(s, v) \leq i$, then v would have been discovered in some layer $L[k]$ where $k \leq i$ (by the inductive hypothesis and the fact that BFS discovers nodes in non-decreasing order of distance). But v was undiscovered when we processed $L[i]$. **Contradiction.** Thus $\text{dist}(s, v) = i + 1$.

($\supseteq$) Let v be a node with $\text{dist}(s, v) = i + 1$. Then there exists a shortest path $s \rightsquigarrow \dots \rightsquigarrow u \rightarrow v$ where $\text{dist}(s, u) = i$. By the induction hypothesis, $u \in L[i]$. When processing u , the algorithm examines edge (u, v) . If v were already discovered, it would be in some layer $L[k]$ with $k \leq i$, implying $\text{dist}(s, v) = k \leq i$. **Contradiction.** Therefore v is undiscovered, and the algorithm adds v to $L[i + 1]$.

Part 2: Path existence $\Leftrightarrow$ appearance in some layer.

($\Rightarrow$) If there is a path from s to t , let $d = \text{dist}(s, t)$. By Part 1, $t \in L[d]$.

($\Leftarrow$) If $t \in L[j]$ for some j , then t was discovered from some node in $L[j - 1]$, which was discovered from $L[j - 2]$, and so on back to $s \in L[0]$. The edges used in discovery form a path from s to t in T . ■

2.6 The BFS Tree and Nontree Edges

Definition: BFS Tree

As BFS executes, it produces a **tree** T rooted at s containing exactly the nodes reachable from s . Node u becomes the **parent** of v if v is discovered while examining u .

The tree T contains exactly one path from s to each reachable node v , and because of layer-by-layer construction, this path is a **shortest path** in G .

Theorem 3.4: BFS Nontree Edge Property

Let T be a BFS tree, let x and y be nodes in T belonging to layers L_i and L_j respectively, and let (x, y) be an edge of G . Then $|i - j| \leq 1$.

Proof

Without loss of generality, assume x is processed before y (or simultaneously if in the same layer). When x is processed, all its neighbors are examined. If y is undiscovered, it is placed in $L[i + 1]$, so $j = i + 1$.

If y is already discovered, it must be in some layer $L[k]$ where $k \leq i + 1$ (since BFS processes layers in order). Also, since (x, y) is an edge and $x \in L_i$:

$$\text{dist}(s, y) \leq \text{dist}(s, x) + 1 = i + 1 \Rightarrow k \leq i + 1$$

Similarly, since (x, y) is an edge:

$$\text{dist}(s, x) \leq \text{dist}(s, y) + 1 = j + 1 \Rightarrow i \leq j + 1 \Rightarrow j \geq i - 1$$

Combining: $i - 1 \leq j \leq i + 1$, so $|i - j| \leq 1$. ■

Interpretation: Nontree edges can only connect nodes within the **same layer** or in **adjacent layers**. They **cannot** connect nodes whose layer numbers differ by 2 or more. If such an edge existed, the deeper node would have been discovered earlier via that edge.

Chapter 3: Depth-First Search (DFS)

3.1 Intuition and the Maze Analogy

Depth-First Search (DFS) corresponds to the strategy you would use if physically walking through a maze:

1. Start at s .
2. Pick the first edge leading out of s to some node v .
3. From v , pick the first edge leading to an unexplored node.
4. Continue going **deeper and deeper** until you reach a "**dead end**" — a node whose all neighbors are explored.
5. When you hit a dead end, **backtrack** to the most recent node that still has an unexplored neighbor, and resume from there.

DFS explores the graph by going as **deeply** as possible along each branch before retreating — hence the name "depth-first."

3.2 Algorithm and Working

Algorithm: DFS(s) — Iterative Stack-Based

```

DFS( $s$ ):
    Initialize  $S$  to be a stack with one element  $s$ 
    While  $S$  is not empty
        Take a node  $u$  from  $S$ 
        If Explored[ $u$ ] = false then
            Set Explored[ $u$ ] = true
            For each edge  $(u, v)$  incident to  $u$ 
                Add  $v$  to the stack  $S$ 
            Endfor
        Endif
    Endwhile
  
```

Initialization: Push the source node s onto stack S . All nodes are initially unexplored.

Main Loop: While the stack is not empty:

1. **Pop** a node u from the stack.
2. If u has not been explored, mark it as `Explored[u] = true`.
3. Push **all neighbors** of u onto the stack.
4. The next iteration pops one of these neighbors (the **last** one pushed), creating a depth-first pattern.

Key Insight: Because we use a stack (LIFO), DFS always extends the **most recently discovered** path. This creates a "deep" exploration pattern rather than the "broad" pattern of BFS.

Definition: DFS Tree

Like BFS, DFS yields a **rooted tree** T on the connected component containing s :

- The root of T is s .
- Node u becomes the **parent** of v if $\text{DFS}(v)$ is invoked directly during the execution of $\text{DFS}(u)$, or equivalently, if v is pushed onto the stack while processing u and v is later marked Explored from that push.

3.3 Visual Walkthrough

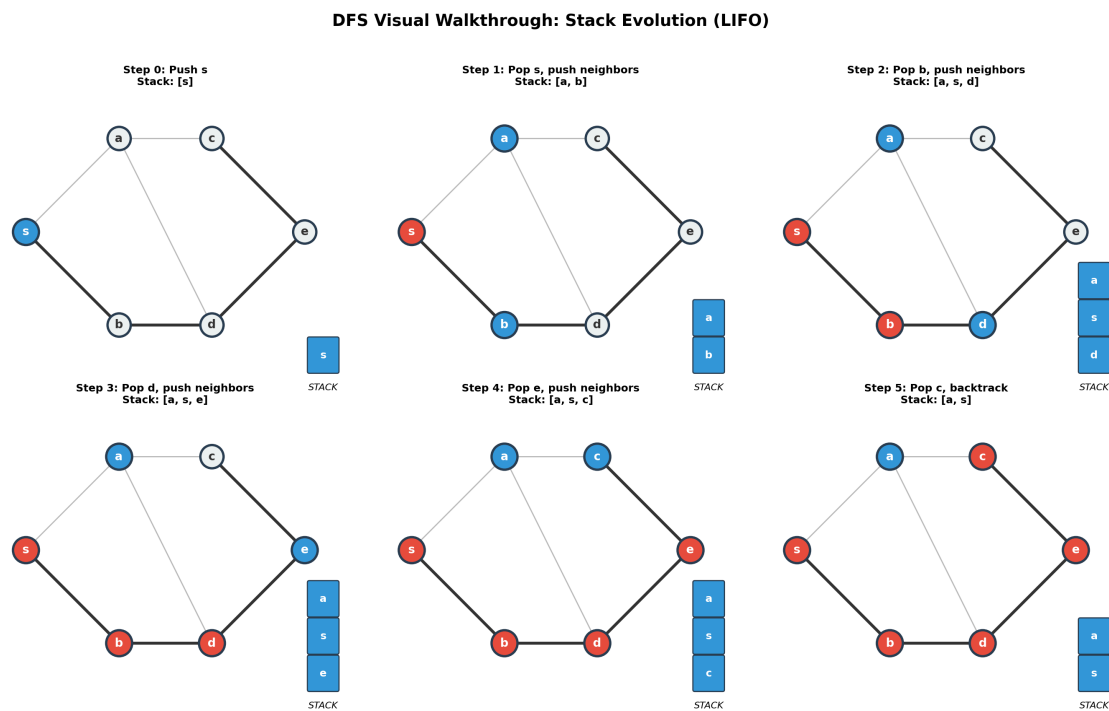


Figure 3: DFS stack evolution starting from node *s*. Red nodes are Explored; blue nodes are on the stack. The stack visualization shows LIFO behavior: the last neighbor pushed is processed first, causing deep exploration.

Execution trace (processing adjacency lists in alphabetical order):

Table 5 DFS Stack Trace

Step	Stack S (top at right)	u popped	Explored	Action
1	[s]	s	$\emptyset$	Push a, b $\rightarrow S = [a, b]$
2	[a, b]	b	{s}	Push s, d $\rightarrow S = [a, s, d]$
3	[a, s, d]	d	{s, b}	Push b, e $\rightarrow S = [a, s, b, e]$
4	[a, s, b, e]	e	{s, b, d}	Push c, d $\rightarrow S = [a, s, b, c, d]$
5	[a, s, b, c, d]	d	{s, b, d, e}	Already explored, skip
6	[a, s, b, c]	c	{s, b, d, e}	Push a, e $\rightarrow S = [a, s, b, a, e]$
7	[a, s, b, a, e]	e	{s, b, d, e, c}	Already explored, skip
8	[a, s, b, a]	a	{s, b, d, e, c}	Push s, c $\rightarrow S = [a, s, b, s, c]$
9–14	Remaining pops all hit already-explored nodes; stack empties. Algorithm terminates.			

DFS Tree edges: $(s, b), (b, d), (d, e), (e, c), (c, a)$. The DFS path goes deep: $s \rightarrow b \rightarrow d \rightarrow e \rightarrow c \rightarrow a$.

3.4 Full Complexity Derivation

Theorem 3.13: DFS Time Complexity

The above implementation of DFS runs in time $O(m + n)$ if the graph is given by adjacency list representation.

Step-by-Step Derivation

Step 1: Node Marking

Each node is marked `Explored` at most once. The check `If Explored[u] = false` is executed once per stack pop. There are at most $O(m)$ pops (see Step 3), so checking costs $O(m)$.

Step 2: When is a node processed?

A node u is first popped and found unexplored exactly once. At that moment, its entire adjacency list $Adj[u]$ is scanned, taking $O(\deg(u))$ time. After being marked `Explored`, subsequent pops of u are skipped in $O(1)$.

Step 3: Stack Push Operations

Node v is pushed onto the stack every time one of its $\deg(v)$ adjacent nodes is explored. Therefore, the total number of pushes is at most:

$$\sum_{u \in V} \deg(u) = 2m = O(m)$$

Since each push corresponds to one eventual pop, the total number of stack operations is $O(m)$.

Step 4: Adjacency List Scanning (the dominant cost)

The total time spent scanning adjacency lists (when nodes are first found unexplored) is:

$$T_{\text{scan}} = \sum_{u \in V} \deg(u) = 2m = O(m)$$

Step 5: Initialization

Initializing `Explored` array and stack: $O(n)$.

Step 6: Total Time

$$T_{\text{total}} = O(n) + O(m) + O(m) = O(m + n)$$

Each of n nodes and m edges is processed a constant number of times. ■

3.5 Structural Theorems on DFS Trees

Theorem 3.6: DFS Descendant Property

For a given recursive call $\text{DFS}(u)$, all nodes that are marked "Explored" between the invocation and end of this recursive call are descendants of u in T .

Proof

In the recursive formulation, when $\text{DFS}(u)$ is called, it marks u as Explored and then recursively calls $\text{DFS}(v)$ for each unexplored neighbor v .

Each such recursive call $\text{DFS}(v)$ marks v and its descendants before returning. Since v is added as a child of u in T , all nodes marked during $\text{DFS}(v)$ are descendants of v , and hence descendants of u .

No nodes outside the subtree of u are marked during $\text{DFS}(u)$ because the algorithm only explores neighbors of nodes in the current call stack. ■

Theorem 3.7: DFS Nontree Edge Property

Let T be a DFS tree, let x and y be nodes in T , and let (x, y) be an edge of G that is not an edge of T . Then one of x or y is an ancestor of the other in T .

Proof

Consider the edge $(x, y) \notin T$. Without loss of generality, assume x was marked Explored **before** y during the DFS execution.

When $\text{DFS}(x)$ was executed, it examined all neighbors of x . Since y was not marked Explored at that time (otherwise (x, y) would be in T), y must have been discovered **later** during the execution of $\text{DFS}(x)$ or during a recursive call initiated from $\text{DFS}(x)$.

If y was discovered during $\text{DFS}(x)$ directly, then y is a **child** of x . If y was discovered during $\text{DFS}(z)$ where z is a descendant of x , then y is a descendant of z , and hence a **descendant** of x . In either case, y is in the subtree rooted at x , meaning x is an **ancestor** of y in T . ■

Theorem 3.12: Iterative DFS Equivalence

The stack-based algorithm implements DFS, visiting nodes in exactly the same order as the recursive DFS procedure (except that each adjacency list is processed in reverse order).

Proof

Both the recursive and iterative versions use a stack. In the recursive version, the call stack implicitly stores the path. When $\text{DFS}(u)$ calls $\text{DFS}(v)$, v is processed before returning to u .

In the iterative version, pushing neighbors onto the stack ensures the **last neighbor pushed** is processed first (LIFO), mimicking the deepest recursive call. The only difference is that the iterative version pushes all neighbors at once, so if adjacency lists are processed in the same order, the iterative version processes them in **reverse** due to stack semantics. Reversing the adjacency list order makes the two identical. ■

Chapter 4: Connected Components

4.1 The Generic Component-Growing Algorithm

BFS and DFS are both instances of a more general algorithm for "exploring" a graph starting from s :

Algorithm: Generic Component Growing

```
R will consist of nodes to which  $s$  has a path  
Initially  $R = \{s\}$   
While there is an edge  $(u, v)$  where  $u \in R$  and  $v \notin R$   
    Add  $v$  to  $R$   
Endwhile
```

Working: The algorithm maintains a set R of reachable nodes. Initially $R = \{s\}$. At each step, if there exists an edge from a node in R to a node outside R , that outside node is added to R . The algorithm terminates when no such edge exists, meaning all edges from R stay within R .

Why is this safe? If there is a path P from s to u (which there must be, since $u \in R$), then there is a path from s to v obtained by first following P and then taking the edge (u, v) . Therefore, v is indeed reachable from s .

Theorem 3.5: Correctness of Component Growing

The set R produced at the end of the algorithm is precisely the connected component of G containing s .

Proof

Let $C(s) = \{v \in V : \text{there is a path from } s \text{ to } v \text{ in } G\}$ be the connected component containing s .

$(R \subseteq C(s))$: By induction on the number of iterations.

- *Base case*: $s \in R$ and $s \in C(s)$.
- *Inductive step*: If v is added to R because (u, v) is an edge with $u \in R$, and $u \in C(s)$ by induction, then there is a path from s to u , and adding edge (u, v) gives a path from s to v . Thus $v \in C(s)$.

$(C(s) \subseteq R)$: Suppose for contradiction that there exists $v \in C(s)$ but $v \notin R$. Since $v \in C(s)$, there is a path $s = v_0, v_1, \dots, v_k = v$ from s to v .

Let v_i be the **first** node on this path that is not in R . Then $i \geq 1$ (since $s \in R$), and $v_{i-1} \in R$. The edge (v_{i-1}, v_i) has one endpoint in R and one outside R .

But the algorithm only terminates when **no such edge exists**. **Contradiction**. Therefore $C(s) \subseteq R$.

Since $R \subseteq C(s)$ and $C(s) \subseteq R$, we have $R = C(s)$. ■

4.2 Finding All Connected Components

To find **all** connected components (not just the one containing s):

Algorithm: All Connected Components

```
Find-All-Components(G):
    Initialize all nodes as unvisited
    While there exists an unvisited node s
        Run BFS or DFS from s to generate its component
        Mark all reached nodes as visited
    Return the set of all components found
```

Why components are disjoint: By definition of connected components, the component found from v will be disjoint from the component of s (and all previously found

components). There are no edges between different connected components.

Running Time: Although we may run BFS/DFS multiple times (once per component), each individual run only spends work on edges and nodes **within that component**. No edge or node is ever processed by more than one invocation.

Complexity Analysis

Each node is visited exactly once across all searches. Each edge is examined exactly once across all searches (twice for undirected graphs, once from each endpoint). Therefore, the total work remains proportional to the total number of nodes plus edges:

$$T_{\text{total}} = O(m + n)$$

4.3 The Disjointness Property

Theorem 3.8: Connected Components are Disjoint

For any two nodes s and t in a graph, their connected components are either identical or disjoint.

Proof

Let $C(s)$ and $C(t)$ be the connected components containing s and t .

Case 1: If there is no path between s and t , then $C(s)$ and $C(t)$ share no common nodes. If they shared a node u , then there would be paths $s \rightsquigarrow u$ and $u \rightsquigarrow t$, implying a path $s \rightsquigarrow t$ by transitivity. **Contradiction.** Thus $C(s) \cap C(t) = \emptyset$.

Case 2: If there is a path between s and t , we show $C(s) = C(t)$.

($C(s) \subseteq C(t)$): Let $v \in C(s)$. Then there is a path from s to v . Since there is a path from s to t , there is a path from t to s (undirected graph). Concatenating path $t \rightsquigarrow s$ with path $s \rightsquigarrow v$ gives a path $t \rightsquigarrow v$. Thus $v \in C(t)$.

($C(t) \subseteq C(s)$): By symmetric argument. Therefore $C(s) = C(t)$. ■

Chapter 5: Testing Bipartiteness

5.1 Definition and 2-Coloring

Definition: Bipartite Graph

A graph $G = (V, E)$ is **bipartite** if its node set V can be partitioned into two disjoint sets X and Y such that **every edge** in E has one end in X and the other end in Y . No edge has both ends in X or both ends in Y .

Coloring analogy: Color all nodes in X **red** and all nodes in Y **blue**. A graph is bipartite if and only if it can be 2-colored so that every edge has one red end and one blue end. This 2-coloring formulation is equivalent to the partition definition.

5.2 BFS-Based Algorithm

Algorithm: Test-Bipartite(s)

```

Test-Bipartite(s):
    Initialize L[0] to be a list with one element s
    Set Color[s] = red
    Set Explored[s] = true
    Set i = 0
    While L[i] is not empty
        Initialize a new empty list L[i + 1]
        For each node u in L[i]
            For each edge (u, v) incident to u
                If Explored[v] = false then
                    Set Explored[v] = true
                    If (i + 1) is even then
                        Set Color[v] = red
                    Else
                        Set Color[v] = blue
                    Endif
                Add v to the list L[i + 1]
            Endif
        Endfor
        Set i = i + 1
    Endwhile
    For each edge (u, v) in the graph
        If Color[u] = Color[v] then
            Return "Not Bipartite"
        Endif
    Endfor
    Return "Bipartite"

```

Working: The algorithm uses BFS to 2-color the graph. Nodes in even layers are colored red; nodes in odd layers are colored blue. After BFS completes, it checks every edge in the graph. If any edge connects two nodes of the same color, the graph is not bipartite.

Key Insight: In a bipartite graph, all edges must go between the two partitions. In BFS terms, edges can only connect nodes in adjacent layers (different parity). An edge within the same layer (same parity) indicates an odd cycle.

5.3 Visual Walkthrough

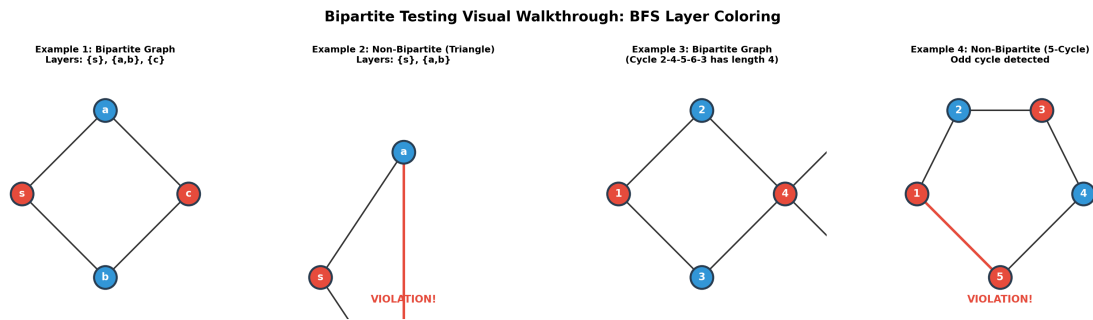


Figure 4: Bipartite testing visual walkthrough. Example 1 (left) shows a valid bipartite graph with proper 2-coloring. Example 2 shows a triangle where the edge (a, b) connects two blue nodes — a violation proving non-bipartiteness. Examples 3 and 4 contrast even and odd cycles.

5.4 Theorems on Bipartiteness

Theorem 3.14: Bipartite Graphs Have No Odd Cycles

If a graph G is bipartite, then it cannot contain an odd cycle.

Proof

Suppose G is bipartite with partition (X, Y) . Consider any cycle $v_1, v_2, \dots, v_k, v_1$.

Without loss of generality, let $v_1 \in X$. Since G is bipartite:

- $v_2 \in Y$ (adjacent to $v_1 \in X$)
- $v_3 \in X$ (adjacent to $v_2 \in Y$)
- $v_4 \in Y$ (adjacent to $v_3 \in X$)

Continuing this alternating pattern: all **odd-indexed** nodes are in X , all **even-indexed** nodes are in Y .

For the cycle to close, v_k must be in Y (to connect back to $v_1 \in X$), so k must be **even**. Therefore every cycle has even length. ■

Theorem 3.15: BFS-Based Bipartite Test is Correct

Let G be a connected graph, and let $L_1, L_2, \dots$ be the layers produced by BFS starting at node s . Then exactly one of the following holds:

- (i) There is no edge joining two nodes of the same layer. In this case G is bipartite.
- (ii) There is an edge joining two nodes of the same layer. In this case G contains an odd-length cycle and cannot be bipartite.

Proof

Mutual Exclusivity: If there is no edge within the same layer, then (ii) is false. If there is an edge within the same layer, then (i) is false. So exactly one holds.

Case (i): Suppose no edge joins two nodes in the same layer. By Theorem 3.4, edges only connect nodes in adjacent layers (or same layer, but we assumed none exist). Color even layers red and odd layers blue. Every edge connects adjacent layers, hence connects red to blue. Therefore G is bipartite. ■

Case (ii): Suppose edge (x, y) joins two nodes in the same layer L_i . Let z be the least common ancestor of x and y in the BFS tree T . Suppose $z \in L_j$ where $j < i$.

In T , there is a path from z to x of length $i - j$, and a path from z to y of length $i - j$. The edge (x, y) closes a cycle: $z \rightsquigarrow x \rightarrow y \rightsquigarrow z$. The length of this cycle is:

$$(i - j) + 1 + (i - j) = 2(i - j) + 1$$

which is **odd**. By Theorem 3.14, G cannot be bipartite. ■

Conclusion: The BFS-based algorithm correctly tests bipartiteness in $O(m + n)$ time because it performs one BFS ($O(m + n)$) plus one scan of all edges ($O(m)$).

Chapter 6: Directed Graphs

6.1 Representing Directed Graphs

The adjacency list representation is adapted for directed graphs as follows:

Directed Adjacency List

Each node u has **two lists** associated with it:

1. **Outgoing edges:** A list of nodes v such that $(u, v) \in E$ (nodes reachable by going one step forward).
2. **Incoming edges:** A list of nodes w such that $(w, u) \in E$ (nodes from which u can be reached by going one step backward).

This dual-list structure allows an algorithm examining node u to read off both the forward-reachable and backward-reachable nodes.

6.2 BFS and DFS in Directed Graphs

BFS and DFS extend naturally to directed graphs by **following edge directions only**:

Table 6 Directed Graph Search Algorithms

Algorithm	Behavior	Computes	Running Time
BFS from s	Follows outgoing edges	Nodes reachable from s	$O(m + n)$
DFS from s	Follows outgoing edges recursively	Nodes reachable from s	$O(m + n)$
BFS from s in G^{rev}	Follows reversed edges	Nodes that can reach to s	$O(m + n)$

The Reversal Trick

Define G^{rev} by reversing every edge: if $(u, v) \in E$, then $(v, u) \in E^{rev}$. Then:

v has a path from s in $G^{rev} \Leftrightarrow v$ has a path to s in G .

6.3 Strong Connectivity

Definition: Mutual Reachability

Two nodes u and v in a directed graph are **mutually reachable** if there is a path from u to v **and** a path from v to u .

Definition: Strongly Connected Graph

A directed graph is **strongly connected** if **every pair** of nodes is mutually reachable.

Theorem 3.16: Transitivity of Mutual Reachability

If u and v are mutually reachable, and v and w are mutually reachable, then u and w are mutually reachable.

Proof

Since u and v are mutually reachable, there exists a path P_1 from u to v and a path P_2 from v to u .

Since v and w are mutually reachable, there exists a path P_3 from v to w and a path P_4 from w to v .

Concatenating P_1 followed by P_3 gives a path from u to w .

Concatenating P_4 followed by P_2 gives a path from w to u .

Therefore u and w are mutually reachable. ■

Corollary: Mutual reachability is an **equivalence relation** (reflexive, symmetric, transitive). Thus it partitions V into equivalence classes.

Algorithm: Testing Strong Connectivity

```

Test-Strongly-Connected(G):
    Pick any node s
    Run BFS (or DFS) in G starting from s
    Run BFS (or DFS) in G_rev starting from s
    If both searches reach every node then
        Return "Strongly Connected"
    Else
        Return "Not Strongly Connected"

```

Proof of Correctness**If both searches reach all nodes:**

- BFS in G from s reaching all nodes means: for every v , there is a path $s \rightsquigarrow v$ in G .
- BFS in G^{rev} from s reaching all nodes means: for every v , there is a path $s \rightsquigarrow v$ in G^{rev} , which means there is a path $v \rightsquigarrow s$ in G .

Therefore, for every v , s and v are mutually reachable.

Now consider any two arbitrary nodes u and w :

- s and u are mutually reachable.
- s and w are mutually reachable.
- By transitivity (Theorem 3.16), u and w are mutually reachable.

Since u and w were arbitrary, **every pair** is mutually reachable. Hence G is strongly connected.

Conversely: If either search misses some node, then there exists a node not reachable from s (or from which s is not reachable), so s is not mutually reachable with that node, and G cannot be strongly connected. ■

Running Time: Two BFS/DFS executions, each $O(m + n)$. Total: $O(m + n)$.

6.4 Strong Components

Definition: Strong Component

The **strong component** containing a node s is the set of all nodes v such that s and v are mutually reachable. It is the maximal set of nodes where every node can reach every other via directed paths.

Computing the strong component of s :

- Run BFS from s in $G \rightarrow$ obtain R_{forward} (nodes reachable **from** s).
- Run BFS from s in $G^{\text{rev}} \rightarrow$ obtain R_{backward} (nodes that can reach **to** s).
- The **intersection** $R_{\text{forward}} \cap R_{\text{backward}}$ is exactly the strong component of s .

Theorem 3.17: Strong Components are Disjoint

For any two nodes s and t in a directed graph, their strong components are either identical or disjoint.

Proof

Let $SCC(s)$ and $SCC(t)$ be the strong components containing s and t .

Case 1: If s and t are mutually reachable, then by Theorem 3.16, any node $u \in SCC(s)$ is mutually reachable with s , and s is mutually reachable with t , so u is mutually reachable with t . Thus $u \in SCC(t)$. Similarly $SCC(t) \subseteq SCC(s)$. Therefore $SCC(s) = SCC(t)$.

Case 2: If s and t are not mutually reachable, then there is no node that is mutually reachable with both (otherwise by transitivity, s and t would be mutually reachable). Thus $SCC(s) \cap SCC(t) = \emptyset$. ■

Chapter 7: DAGs and Topological Ordering

7.1 Directed Acyclic Graphs

Definition: DAG

A **directed acyclic graph** (DAG) is a directed graph that contains **no directed cycles**.

Contrast with undirected acyclic graphs: In undirected graphs, acyclicity implies simplicity: each component is a tree with $n - c$ edges. Directed graphs behave very differently. A DAG can be dense: the graph on nodes $\{1, 2, \dots, n\}$ with edges (i, j) for all $i < j$ has $\binom{n}{2} = \Theta(n^2)$ edges and **no directed cycles** (every edge goes from smaller to larger index).

Why DAGs are ubiquitous: DAGs naturally encode **precedence relations**. If task i must be done before task j , add edge (i, j) . If the graph contained a cycle, the precedence relation would be **internally contradictory** — no task in the cycle could ever be done first.

7.2 Topological Ordering

Definition: Topological Ordering

A **topological ordering** of a directed graph G with n nodes is an ordering $v_1, v_2, \dots, v_n$ such that for **every directed edge** $(v_i, v_j) \in E$, we have $i < j$. Every edge points "forward" in the ordering.

Scheduling interpretation: A topological ordering provides a valid linear sequence for performing all tasks. When we reach task v_j , every prerequisite task v_i appears earlier and has already been completed.

Theorem 3.18: Topological Ordering Implies DAG

If G has a topological ordering, then G is a DAG.

Proof

Suppose for contradiction that G has a topological ordering and contains a directed cycle $C = (v_{i_1}, v_{i_2}, \dots, v_{i_k}, v_{i_1})$.

By the topological ordering property:

- $(v_{i_1}, v_{i_2}) \in E \Rightarrow i_1 < i_2$
- $(v_{i_2}, v_{i_3}) \in E \Rightarrow i_2 < i_3$
- $\dots$
- $(v_{i_k}, v_{i_1}) \in E \Rightarrow i_k < i_1$

Chaining these: $i_1 < i_2 < \dots < i_k < i_1$. **Contradiction.** Therefore G contains no directed cycles. ■

Theorem 3.19: Every DAG Has a Source Node

In every DAG G , there is at least one node with no incoming edges (in-degree 0).

Proof

Suppose for contradiction that every node has at least one incoming edge. Start from any node v_1 and follow an incoming edge to v_2 , then from v_2 follow an incoming edge to v_3 , and so on.

Since there are only n nodes, by the **pigeonhole principle**, some node must repeat within $n + 1$ steps, creating a directed cycle. This contradicts G being a DAG. ■

Theorem 3.20: Every DAG Has a Topological Ordering

If G is a DAG, then G has a topological ordering.

Proof (by Induction on n)

Base Case ($n = 1$): The single node is trivially a topological ordering.

Inductive Step: Assume every DAG with $k < n$ nodes has a topological ordering. Let G be a DAG with n nodes.

By Theorem 3.19, G has a node v with no incoming edges. Remove v to obtain $G' = G - \{v\}$. G' is also a DAG (removing nodes cannot create cycles). G' has $n - 1$ nodes, so by the induction hypothesis, G' has a topological ordering $v_1, v_2, \dots, v_{n-1}$.

Place v first, followed by the topological ordering of G' : $v, v_1, v_2, \dots, v_{n-1}$.

Since v has no incoming edges, all edges involving v are outgoing, so they go from v to nodes in G' , satisfying $i < j$. All other edges are within G' and respect its topological ordering. Therefore this is a topological ordering of G . ■

7.3 Algorithm and Visual Walkthrough

Algorithm: Topological Sort

```

Topological-Sort(G):
    Compute in-degree for all nodes
    Initialize queue Q with all nodes of in-degree 0
    Initialize empty list Order
    While Q is not empty
        Remove node v from Q
        Append v to Order
        For each edge (v, w) in G
            Decrement in-degree of w
            If in-degree of w becomes 0 then
                Add w to Q
            Endif
        Endfor
    Endwhile
    If Order contains all n nodes then
        Return Order
    Else
        Return "Graph contains a cycle"
  
```

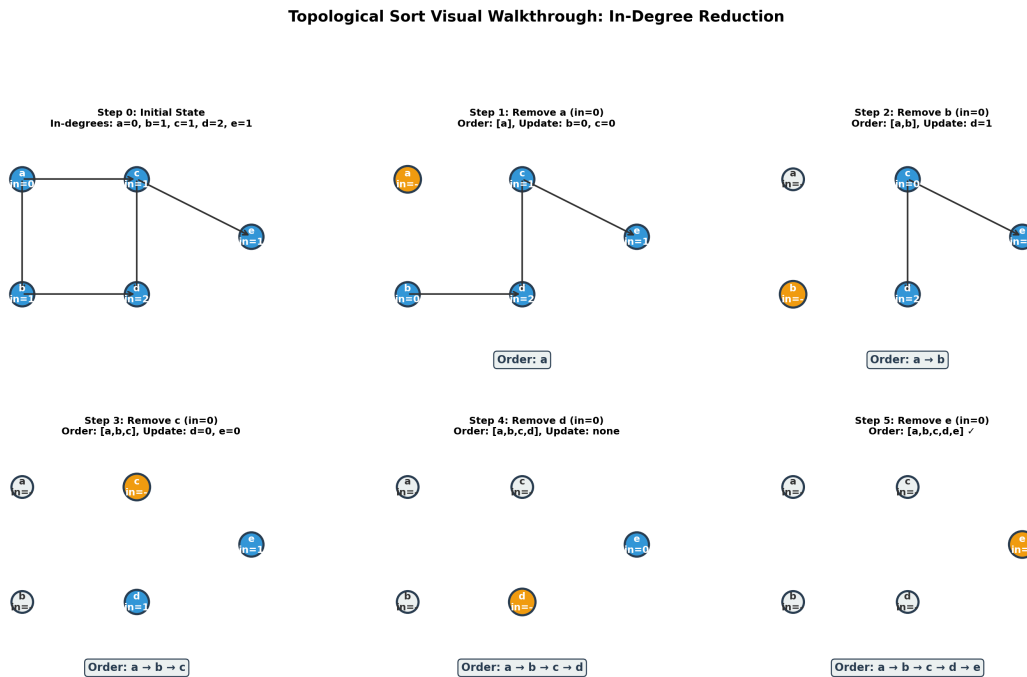


Figure 5: Topological sort using in-degree reduction. At each step, a node with in-degree 0 is removed (highlighted in orange), its outgoing edges are deleted, and in-degrees are updated. The final order $a \rightarrow b \rightarrow c \rightarrow d \rightarrow e$ is a valid topological ordering.

Complexity Analysis

Naive implementation: Scanning all nodes to find in-degree 0 at each step takes $O(n^2)$ time.

Efficient implementation (using a queue):

- Each node is removed from the queue exactly once: $O(n)$ queue operations.
- Each edge is examined exactly once (when its tail is removed): $O(m)$ work.
- In-degree updates are $O(1)$ per edge.

Total time: $O(m + n)$.

Chapter 8: Dijkstra's Algorithm

8.1 Overview and Invariant

Dijkstra's algorithm finds shortest paths from a source node s to all other nodes in a graph with **non-negative edge lengths**. It maintains a set S of **explored** nodes and a distance estimate $d(u)$ for each $u \in S$, which is the **true** shortest path distance.

Invariant

At each step, S contains nodes whose shortest path distance from s is known exactly. For each $u \in S$, $d(u)$ is the length of the shortest s - u path.

8.2 Algorithm and Working

Algorithm: Dijkstra's Algorithm

```
Dijkstra(G, l):
  Let S be the set of explored nodes
  For each  $u \in S$ , store distance  $d(u)$ 
  Initially  $S = \{s\}$  and  $d(s) = 0$ 
  While  $S \neq V$ 
    Select a node  $v \notin S$  with at least one edge from  $S$  for which
       $d'(v) = \min_{e=(u,v): u \in S} (d(u) + l_e)$ 
    is as small as possible
    Add  $v$  to  $S$  and define  $d(v) = d'(v)$ 
  EndWhile
```

Working:

1. **Initialization:** $S = \{s\}$, $d(s) = 0$. This is correct since the shortest path from s to itself has length 0.
2. **Main Loop:** At each iteration:
 - For each node $v \notin S$ with at least one edge from S , compute $d'(v) = \min_{(u,v): u \in S} [d(u) + l(u, v)]$. This represents the shortest path from s to v that stays

entirely within S except for the last edge.

- Select the node v with minimum $d'(v)$ among all such nodes.
- Add v to S and set $d(v) = d'(v)$.

Key Insight: Because all edge lengths are non-negative, the shortest path to v cannot pass through any unexplored node (such a path would be at least as long as $d'(v)$). Therefore $d'(v)$ is indeed the true shortest path distance.

8.3 Visual Walkthrough

Dijkstra's Algorithm Visual Walkthrough: Priority Queue Evolution

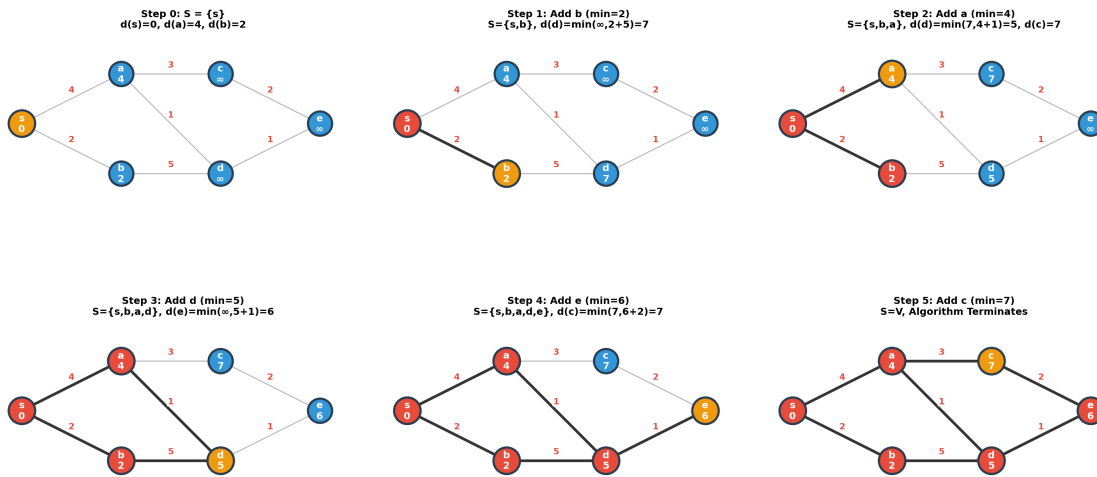


Figure 6: Dijkstra's algorithm execution. Red nodes are in S (distances known). Blue nodes are candidates with distance estimates shown. Orange node is the one being added in each step. Edge weights are shown in red. The algorithm greedily adds the node with minimum $d'(v)$ at each step.

Table 7 Dijkstra Execution Trace

Step	S	Candidates	$d'(v)$ Calculation	Min	Added
0	{s}	a, b	$d'(a) = 0 + 4 = 4, d'(b) = 0 + 2 = 2$	b (2)	b, $d(b) = 2$
1	{s, b}	a, d	$d'(a) = 4, d'(d) = 2 + 5 = 7$	a (4)	a, $d(a) = 4$
2	{s, b, a}	c, d	$d'(c) = 4 + 3 = 7, d'(d) = \min(7, 4 + 1) = 5$	d (5)	d, $d(d) = 5$
3	{s, b, a, d}	c, e	$d'(c) = 7, d'(e) = 5 + 1 = 6$	e (6)	e, $d(e) = 6$
4	{s, b, a, d, e}	c	$d'(c) = \min(7, 6 + 2) = 7$	c (7)	c, $d(c) = 7$
5	All nodes	—	—	—	Done

Resulting Shortest Path Tree:

- $s \rightarrow b$ (length 2)
- $s \rightarrow a$ (length 4)
- $s \rightarrow a \rightarrow d$ (length 5)
- $s \rightarrow a \rightarrow c$ (length 7)
- $s \rightarrow a \rightarrow d \rightarrow e$ (length 6)

8.4 Full Complexity Derivation

Theorem 4.15: Dijkstra Time Complexity

Using a priority queue, Dijkstra's Algorithm can be implemented to run in $O(m)$ time plus the time for n ExtractMin and m ChangeKey (DecreaseKey) operations.

Step-by-Step Derivation

Step 1: Node Extractions

Each node is added to S exactly once. Each addition requires one **ExtractMin** from the priority queue. Total ExtractMin operations: n .

Step 2: Key Updates

For each edge (u, v) , when u is added to S , we may need to update the key (distance estimate) of v . This happens at most **once per edge**, giving m **ChangeKey** (DecreaseKey) operations total.

Step 3: All Other Operations

Computing d' values, updating S , and initializing data structures are all $O(1)$ per edge/node. Total: $O(m + n)$.

Step 4: Total Abstract Running Time

$$T_{\text{total}} = O(m) + n \cdot T_{\text{ExtractMin}} + m \cdot T_{\text{ChangeKey}}$$

Step 5: Concrete Implementations

Priority Queue	ExtractMin	ChangeKey	Total Time
Binary Heap	$O(\log n)$	$O(\log n)$	$O(m \log n)$
Fibonacci Heap	$O(\log n)$ amortized	$O(1)$ amortized	$O(m + n \log n)$
Array (linear scan)	$O(n)$	$O(1)$	$O(n^2)$

For dense graphs where $m \approx n^2$, the array implementation achieves $O(n^2)$, matching the binary heap. For sparse graphs, the binary heap at $O(m \log n)$ is preferred. ■

8.5 Proof of Correctness

Theorem 4.14: Dijkstra's Distances Are Optimal

Consider the set S at any point in the algorithm's execution. For each $u \in S$, the path P_u is a shortest s - u path.

Proof (by Induction on $|S|$)

Base Case: $|S| = 1$, so $S = \{s\}$. The path P_s has length 0, which is optimal.

Inductive Step: Assume the theorem holds for $|S| = k$. Let v be the next node added to S , with:

$$d(v) = d'(v) = d(u) + l(u, v)$$

for some $u \in S$.

Suppose for contradiction that there exists a **shorter** s - v path P . Let y be the **first** node on P that is not in S , and let x be the node just before y on P (so $x \in S$).

The subpath from s to x has length at least $d(x)$ (by induction, $d(x)$ is the shortest distance). The edge (x, y) has length $l(x, y) \geq 0$. So the length of P up to y is at least:

$$d(x) + l(x, y) = d'(y)$$

Since all edge lengths are non-negative, the length of P from y to v is ≥ 0 . Therefore:

$$\text{length}(P) \geq d'(y) \geq d'(v)$$

where the last inequality follows because the algorithm chose v as the node with **minimum** d' value among all candidates. But $d'(v) = d(v)$, so:

$$\text{length}(P) \geq d(v)$$

This contradicts P being shorter than $d(v)$. Therefore P_v is a shortest s - v path. ■

Chapter 9: Minimum Spanning Trees

9.1 The MST Problem

Problem: Minimum Spanning Tree (MST)

Given a connected, undirected graph $G = (V, E)$ with edge costs c_e , find a subset $T \subseteq E$ such that (V, T) is connected and the total cost $\sum_{e \in T} c_e$ is minimized.

Theorem 4.16: The MST is a Tree

Let T be a minimum-cost solution to the network design problem. Then (V, T) is a tree.

Proof

Suppose (V, T) is connected but not a tree. Then T contains a cycle C . Remove any edge e from C . The graph remains connected (removing an edge from a cycle does not disconnect).

The new set $T' = T \setminus \{e\}$ has strictly lower cost since $c_e > 0$ (or even if $c_e = 0$, we can remove it without increasing cost). This contradicts the minimality of T .

Therefore (V, T) is acyclic and connected, hence a tree. ■

9.2 The Cut Property

Definition: Cut

A **cut** in a graph is a partition of V into two non-empty sets S and $V \setminus S$. An edge **crosses** the cut if it has one end in S and one end in $V \setminus S$.

Theorem 4.17: The Cut Property

Assume all edge costs are distinct. Let S be any non-empty proper subset of V , and let $e = (v, w)$ be the minimum-cost edge with one end in S and the other in $V \setminus S$. Then every minimum spanning tree contains e .

Proof

Suppose for contradiction that there exists an MST T that does not contain $e = (v, w)$, where $v \in S$ and $w \in V \setminus S$.

Since T is a spanning tree, there is a **unique path** P in T from v to w . This path must cross the cut $(S, V \setminus S)$ at some edge $e' = (x, y)$ where $x \in S$ and $y \in V \setminus S$.

Consider $T' = T \cup \{e\} \setminus \{e'\}$. Adding e to T creates a cycle (the path P plus e). Removing e' breaks the cycle and restores a tree.

Since e is the **minimum-cost** edge crossing the cut and all costs are distinct:

$$c_e < c_{e'}$$

Therefore:

$$\text{cost}(T') = \text{cost}(T) + c_e - c_{e'} < \text{cost}(T)$$

This contradicts T being minimum. Therefore **every** MST contains e . ■

9.3 The Cycle Property

Theorem 4.20: The Cycle Property

Assume all edge costs are distinct. Let C be any cycle in G , and let e be the most expensive edge in C . Then e does not belong to any minimum spanning tree.

Proof

Suppose for contradiction that an MST T contains $e = (v, w)$, the most expensive edge in cycle C .

Removing e from T disconnects T into two components (since T is a tree). The cycle C must contain another edge e' that reconnects these two components (since C is a cycle, there is another path from v to w).

Consider $T' = T \setminus \{e\} \cup \{e'\}$. Since e is the most expensive edge in C :

$$c_{e'} < c_e$$

Therefore:

$$\text{cost}(T') = \text{cost}(T) - c_e + c_{e'} < \text{cost}(T)$$

This contradicts T being minimum. Therefore e does not belong to any MST. ■

9.4 Prim's Algorithm

Prim's algorithm grows an MST from a starting node, adding the cheapest edge that connects the growing tree to a node outside it. It is analogous to Dijkstra's algorithm but with edge costs instead of path lengths.

Algorithm: Prim's Algorithm

```

Prim(G):
    Initialize S to be an empty set
    Initialize Q to be a priority queue
    For each node v in V
        Set a[v] = infinity
        Set edgeTo[v] = null
        Insert v into Q with key a[v]
    Endfor
    Choose an arbitrary starting node s
    Set a[s] = 0
    ChangeKey(Q, s, a[s])
    While Q is not empty
        Take a node u from Q using ExtractMin(Q)
        Add u to S
        For each edge (u, v) incident to u
            If v is not in S then
                If cost(u, v) < a[v] then
                    Set a[v] = cost(u, v)
                    Set edgeTo[v] = u
                    ChangeKey(Q, v, a[v])
                Endif
            Endif
        Endfor
    Endwhile
    Return edgeTo

```

Working: Each node v has a key $a[v]$ representing the minimum cost to connect v to the growing tree S . When we extract v with minimum $a[v]$, we add the cheapest possible connection from S to $V \setminus S$.

Prim's Algorithm Visual Walkthrough: MST Growth

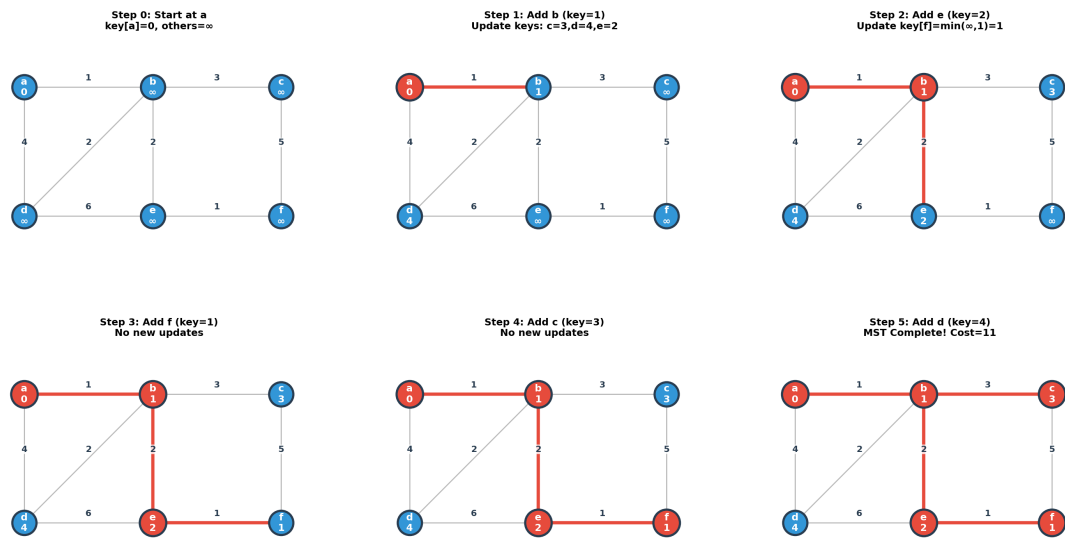


Figure 7: Prim's algorithm starting from node a . Red nodes are in S ; keys show minimum connection cost. At each step, the node with minimum key is extracted and its neighbors' keys are updated. Total MST cost = 11.

Theorem 4.19: Prim's Algorithm Produces an MST

Prim's Algorithm produces a minimum spanning tree of G .

Proof

Prim's algorithm maintains a set $S \subseteq V$ on which a partial spanning tree has been constructed. In each iteration, it adds a node v and edge e that minimize:

$$\min_{e=(u,v):u \in S} c_e$$

By definition, e is the **cheapest edge** with one end in S and the other in $V \setminus S$. By the Cut Property (Theorem 4.17), e belongs to **every** minimum spanning tree.

Since Prim's only adds edges that belong to every MST, and it produces a spanning tree (connects all nodes, acyclic), the result is a minimum spanning tree. ■

Theorem 4.22: Prim's Complexity

Using a priority queue, Prim's Algorithm runs in $O(m)$ time plus the time for n ExtractMin and m ChangeKey operations.

Step-by-Step Derivation**Step 1: Initialization**

Setting up keys and inserting into queue: $O(n)$.

Step 2: Main Loop

n iterations. Each performs one ExtractMin: total n **ExtractMin** operations.

Step 3: Edge Processing

Each edge (u, v) is examined at most twice (once from each endpoint). When examining (u, v) , if $v \notin S$, we may perform a ChangeKey: at most m **ChangeKey** operations total.

Step 4: Total

$$T_{\text{total}} = O(m) + n \cdot T_{\text{ExtractMin}} + m \cdot T_{\text{ChangeKey}}$$

With a binary heap: $O(m \log n)$. With a Fibonacci heap: $O(m + n \log n)$. ■

9.5 Reverse-Delete Algorithm

Algorithm: Reverse-Delete

```
Reverse-Delete(G):
    Sort all edges in decreasing order of cost
    T = E (all edges)
    For each edge e in decreasing order
        If removing e does not disconnect (V, T) then
            Remove e from T
        Endif
    Endfor
    Return T
```

Working: The algorithm processes edges from most expensive to least expensive. For each edge, if removing it does **not** disconnect the graph, it is removed. Otherwise, it is kept because it is a bridge.

Theorem 4.21: Reverse-Delete Produces an MST

The Reverse-Delete Algorithm produces a minimum spanning tree of G .

Proof

Consider any edge $e = (v, w)$ removed by Reverse-Delete. At the time e is removed, it lies on a cycle C (otherwise removing it would disconnect the graph).

Since edges are processed in **decreasing** order, e is the **most expensive** edge on C .

By the Cycle Property (Theorem 4.20), the most expensive edge on any cycle is **not** in any MST. Therefore e does not belong to any minimum spanning tree.

Since Reverse-Delete only removes edges that are not in any MST, and the remaining edges form a spanning tree, the result is a minimum spanning tree. ■

Summary: Key Facts and Complexity Table

Table 8 Algorithm Complexities Summary

Algorithm	Time Complexity (Adj. List)	Space	Key Data Structure
BFS	$O(m + n)$	$O(m + n)$	Queue (FIFO)
DFS	$O(m + n)$	$O(m + n)$	Stack (LIFO / recursion)
Dijkstra (binary heap)	$O(m \log n)$	$O(m + n)$	Priority Queue
Dijkstra (Fibonacci heap)	$O(m + n \log n)$	$O(m + n)$	Priority Queue
Prim (binary heap)	$O(m \log n)$	$O(m + n)$	Priority Queue
Prim (Fibonacci heap)	$O(m + n \log n)$	$O(m + n)$	Priority Queue
Topological Sort	$O(m + n)$	$O(m + n)$	Queue (in-degree tracking)
Bipartite Test	$O(m + n)$	$O(m + n)$	BFS layers + coloring
Strong Connectivity Test	$O(m + n)$	$O(m + n)$	Two BFS/DFS passes

Table 9 Key Theorems Summary

Theorem	Statement	Significance
3.1	n -node tree has $n - 1$ edges	Characterizes trees
3.2	Any two of {connected, acyclic, $n - 1$ edges} implies the third	Equivalent tree definitions
3.3	BFS layers contain nodes at exact distance j from s	BFS correctness for shortest paths
3.4	BFS tree edges connect nodes at most 1 layer apart	Structure of BFS trees
3.5	Component exploration finds exact connected component	Correctness of connectivity algorithms
3.6	DFS marks only descendants during a recursive call	DFS tree structure
3.7	Non-tree edges in DFS connect ancestor-descendant	Fundamental DFS property
3.8	Connected components are identical or disjoint	Partition property
3.9	$\sum \deg(v) = 2m$	Handshaking lemma
3.10	Adjacency matrix: $O(n^2)$; Adj. list: $O(m + n)$	Space trade-offs
3.11	BFS runs in $O(m + n)$ time	Linear-time traversal
3.13	DFS runs in $O(m + n)$ time	Linear-time traversal
3.14	Bipartite graphs have no odd cycles	Necessary condition
3.15	Same-layer edge $\Leftrightarrow$ odd cycle $\Leftrightarrow$ non-bipartite	BFS bipartite test is correct
3.16	Mutual reachability is transitive	SCCs form equivalence classes
3.17	Strong components are identical or disjoint	Partition into SCCs
3.18	Topological ordering $\Rightarrow$ DAG	Necessary condition
3.19	Every DAG has a node with in-degree 0	Existence for topological sort

3.20	DAG $\Rightarrow$ has topological ordering	Sufficient condition
4.14	Dijkstra's distances are optimal when added to S	Dijkstra correctness
4.15	Dijkstra with priority queue: $O(m) + n \text{ ExtractMin} + m \text{ ChangeKey}$	Complexity framework
4.16	MST solution is always a tree	Justifies "tree" terminology
4.17	Cut Property: minimum crossing edge is in every MST	Foundation of MST algorithms
4.19	Prim's algorithm produces an MST	Prim's correctness
4.21	Reverse-Delete produces an MST	Reverse-Delete correctness
